COURSE I: Neural Networks and Deep learning

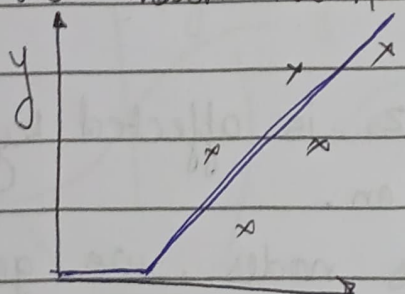
deep learning → training neural network

Neural Network:

let's say we have some data like

x	y
x_1	y_1
x_2	y_2
x_3	y_3
⋮	⋮

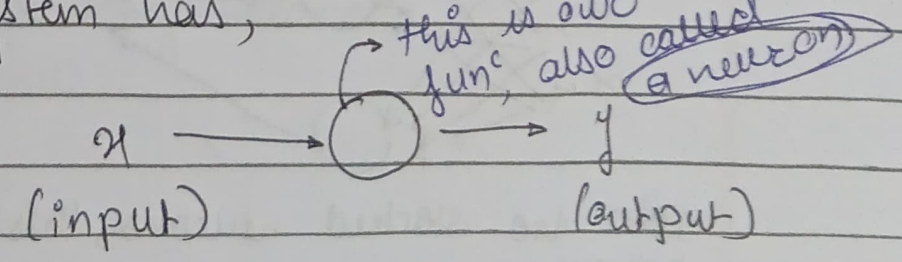
if we mark them on a graph, it goes



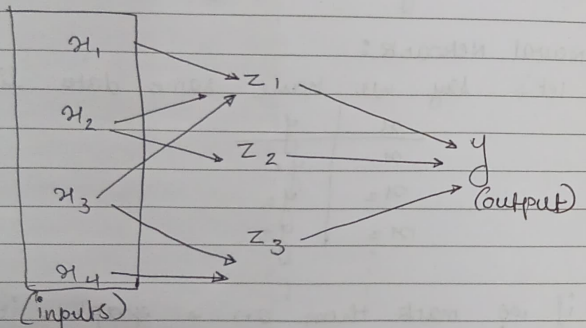
we need to predict the output, given the input so we decide a func^c ⇒

x in this case its kind of like a linear function but it steeps to 0 for some values.

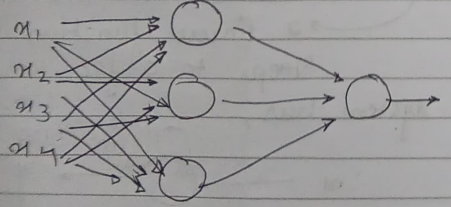
So our system has,



now suppose we have multiple factors that affect our output so we map them like,



now we know, z_1 is affected by x_1, x_2 and x_3 and so on.
considering them as nodes, we get



now we stacked neurons, this is a neural network

If we give all the inputs to a neural network and it maps and decides the nodes on its own, do we really need to clean the datasets and test for compatibility like say a DDI is targetting a protein but it is not in the KEGG & Reactome so NN will ignore it.

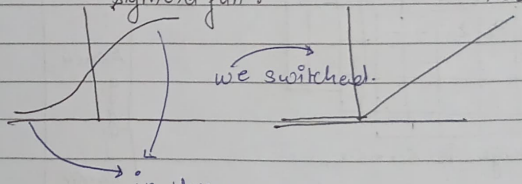
~~we~~ give all the in
so essentially we are
to decide what the
generate functions.

supervised learning

standard NN
CNN → image processing
sequence data → RNN

Algorithic advances :

sigmoid func :



in these areas, due to the slope there is no gradient hence the learnings become extremely slow.

ReLU func :
(rectified linear unit)

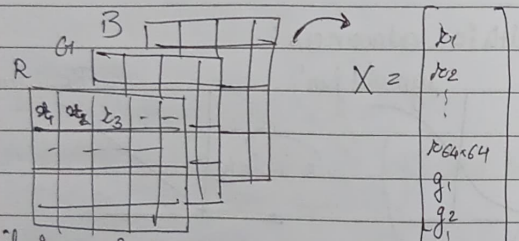
Week 1: Introduction
Week 2: Basics of Neural Network programming
Week 3: one hidden layer Neural Networks
Week 4: Deep Neural Networks

rise of Deep learning:
i) Data
ii) Computations
iii) Algorithm.

Binary classification:

input (x) → y {0, 1}

for images



$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{64\times64} \\ g_1 \\ g_2 \\ \vdots \\ g_{64\times64} \\ b_1 \\ b_2 \\ \vdots \\ b_{64\times64} \end{bmatrix}$

if image is 64×64.

(x, y) ; $x \in \mathbb{R}^{n_x}$, $y \in \{0, 1\}$

m training ex: $\{(x^1, y^1), (x^2, y^2), \dots\}$

$X = \begin{bmatrix} | & | & & | \\ x^1 & x^2 & \cdots & x^m \\ | & | & & | \end{bmatrix}$ $n_x \times m$

$X \in \mathbb{R}^{n_x \times m}$

X.shape = (n_x, m) → python

Logistic Regression:

given x (input), we need $\hat{y} = P(y=1 \mid x)$

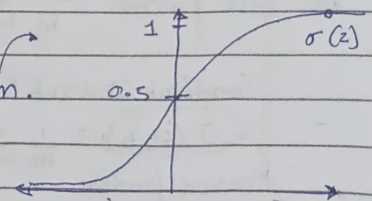
$x \in \mathbb{R}^{n_x}$

Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

output: $\hat{y} = w^T x + b$ → we don't use this, because it will give values > 1, but we need probability of $y=1$ given input x.

we use:

$\hat{y} = \sigma(w^T x + b)$ → sigmoid function.



$\sigma(z) = \frac{1}{1+e^{-z}}$

if $z >> 0$ $\sigma(z) \approx \frac{1}{1+0} = 1$

if $z << 0$ $\sigma(z) = \frac{1}{1+e^{-z}} \approx \frac{1}{1+\infty} \approx 0$

To get w and b, we need a cost function.

~~not a~~ $\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$ good loss function.

because of optentation problem.

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1-y) \log(1-\hat{y}))$$

→ we want the loss func^c to be as small as possible because it denotes how accurate our $\hat{y}$ is given y.

if $y=1$: $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$ ← we want $\log \hat{y}$ ↑, $\hat{y}$ ↑

if $y=0$: $\mathcal{L}(\hat{y}, y) = -\log(1-\hat{y})$ ← we want $\log(1-\hat{y})$ ↑, $\hat{y}$ ↓

Cost function :

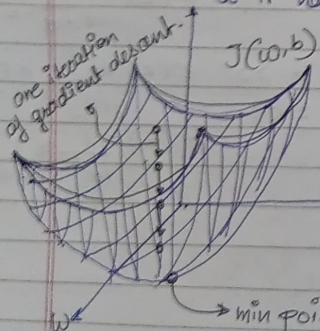
we find w & b that minimizes our cost function.

$$J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)}) \right]$$

Gradient Descent :

$J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

convex function → so it has a minimum.



To get the values of w, b where cost func^c is min. we initialize w, b to $(0, 0)$ (random values work too) → then one iteration of gradient descent brings our point in the downward steep direction.

Repeat :

$$\{ w := w - \alpha \frac{dJ(w,b)}{dw} \quad \text{← learning rate}$$

$$\}$$

$$\{ b := b - \alpha \frac{dJ(w,b)}{db}$$

$$\}$$

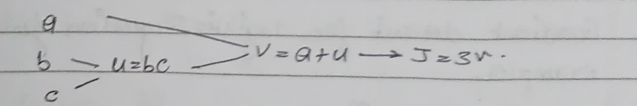
Computation Graph :

$J(a,b,c) = 3(a+bc)$

$u = bc$

$v = a + u$

$J = 3v$

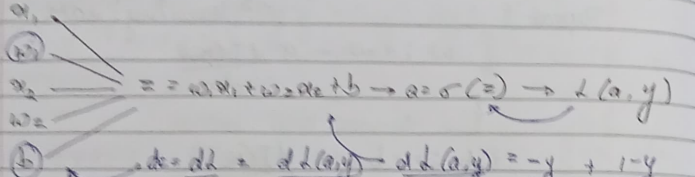


Gradient Descent for logistic Regression :

$z = w^T x + b$

$\hat{y} = a = \sigma(z)$

$\mathcal{L}(a, y) = -(y \log(a) + (1-y) \log(1-a))$



$$\Rightarrow \frac{d\mathcal{L}}{da} \cdot \frac{da}{dz}$$

$$= a - y$$

$$\frac{d\mathcal{L}}{dw_1} = x_1 \cdot dz$$

$$dw_2 = x_2\, dz$$

$$db = dz$$

Gradient descent for Logistic Regression for m examples.

$$J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(a^{(i)}, y^{(i)})$$

$$a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(w^T x^{(i)} + b)$$

$$\frac{\partial}{\partial w_1} J(w,b) = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial}{\partial w_1} \mathcal{L}(a^{(i)}, y^{(i)})$$

** one iteration for gradient descent

Code]

$J = 0, dw_1 = 0, dw_2 = 0, db = 0$

for $i = 1$ to m

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J \mathrel{+}= -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$$

$$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$$

$$db \mathrel{+}= dz^{(i)}$$

$J /= m$

$dw_1 /= m$, $dw_2 /= m$, $db /= m$

$$dw_1 = \frac{\partial J}{\partial w_1}$$

$$w_1 := w_1 - \alpha\, dw_1$$

$$w_2 := w_2 - \alpha\, dw_2$$

$$b := b - \alpha\, db$$

Vectorisation

getting rid of explicit for loops.

vectorized:

$$z = np.dot(w, x) + b$$

$w^T x$

np commands:

u = np.exp(v)
np.log(v)
np.abs(v)
np.maximum(v, 0)
v**2
1/v

vectorization of gradient descent for logistic regression.

initialize as vectors

$dw = np.zeros((n_x, 1))$

$J = 0$, $dw_1 = 0$, $dw_2 = 0$, $db = 0$

for i = 1 to m:

$z^{(i)} = w^T x^{(i)} + b$

$a^{(i)} = \sigma(z^{(i)})$

$J \mathrel{+}= -[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-y^{(i)})]$

$dz^{(i)} = a^{(i)} (1-a^{(i)})$

remove for loop

for j = 1 to m

$dw_1 \mathrel{+}= x_1^{(i)} dz^{(i)}$

$dw_2 \mathrel{+}= x_2^{(i)} dz^{(i)}$

$db \mathrel{+}= dz^{(i)}$

$\rightarrow dw \mathrel{+}= x^{(i)} dz^{(i)}$

$J = J/m$, $dw_1 = dw_1/m$, $dw_2 = dw_2/m$, $db = db/m$

$dw /= m$

Vectorizing logistic Regression:

$z = np.dot(w, x) + b$

$a = \sigma(z)$

Broadcasting in Python.

if u have a matrix, to sum columns → cal = A.sum (axis=0)

Percentage = 100*A / cal.reshape(1, 4) → to sum vertically axis = 1

if we add a matrix (m, n) to (1, n) matrix, python copies it to (m, n).

∴ (m, n) $\overset{+}{*}$ (1, n) ⇝ (m, n).

/ (m, 1) ⇝ (m, n)

Python / numpy vectors.

* don't use rank 1 array

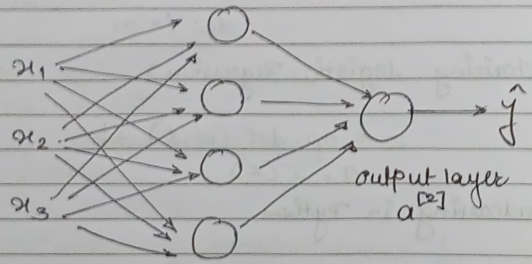
column vector

`a = np.random.randn(5)` → `a.shape = (5,1)`

`a = np.random.randn(1,5)` → `a.shape = (1,5)`

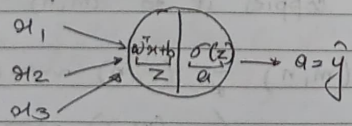
row vector.

Neural Networks



activation layer

each node :



$[i]$ → layer

a_i → node in layer

$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]}, \quad a_1^{[1]} = \sigma(z_1^{[1]})$$
$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]}, \quad a_2^{[1]} = \sigma(z_2^{[1]})$$
$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]}, \quad a_3^{[1]} = \sigma(z_3^{[1]})$$
$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]}, \quad a_4^{[1]} = \sigma(z_4^{[1]})$$

vectorisation,

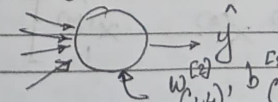
$$z^{[1]} = \begin{bmatrix} — w_1^{[1]T} — \\ — w_2^{[1]T} — \\ — w_3^{[1]T} — \\ — w_4^{[1]T} — \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix}$$

$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ \vdots \\ a_4^{[1]} \end{bmatrix} = \sigma(z^{[1]})$$

given input x :

$$z^{[1]} = w^{[1]} + b^{[1]}$$
$$(4,1) \quad (3,1) \quad (4,1)$$
$$a^{[1]} = \sigma(z^{[1]})$$
$$(4,1) \quad (4,1)$$

for output node :



vectorizing across multiple examples

$x \longrightarrow a^{[2]} = \hat{y}$

$x^{(1)} \longrightarrow a^{[2](1)} = \hat{y}^{(1)}$

$x^{(2)} \longrightarrow a^{[2](2)} = \hat{y}^{(2)}$

$\vdots \qquad \vdots$

$x^{(m)} \longrightarrow a^{[2](m)} = \hat{y}^{(m)}$

$a^{[2](i)}$ ← example i

↑ layer 2

for $i = 1$ to m

$z^{[1](i)} = w^{[1]} x^{(i)} + b^{[1]}$

$a^{[1](i)} = \sigma(z^{[1](i)})$

$z^{[2](i)} = w^{[2]} a^{[1](i)} + b^{[2]}$

$a^{[2](i)} = \sigma(z^{[2](i)})$

vectorizing:

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$Z^{[1]} = w^{[1]} X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = w^{[2]} A^{[1]} + b^{[2]}$$

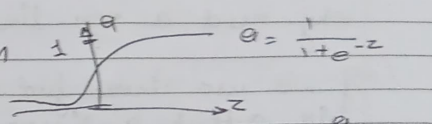
$$A^{[2]} = \sigma(Z^{[2]})$$

$$Z^{[1]} = \begin{bmatrix} | & | & & | \\ z^{[1](1)} & z^{[1](2)} & \cdots & z^{[1](m)} \\ | & | & & | \end{bmatrix}$$

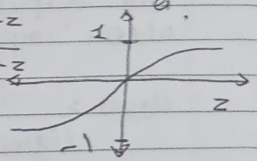
$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{[1](1)} & a^{[1](2)} & \cdots & a^{[1](m)} \\ | & | & & | \end{bmatrix}$$

Activation function

⇒ Sigmoid function

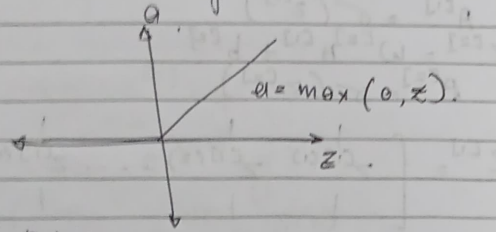


⇒ $g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

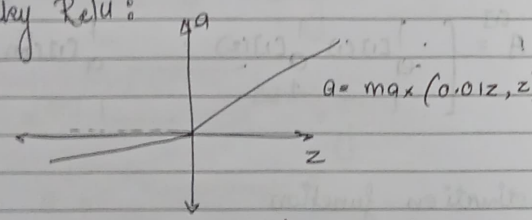


↳ works better cause the mean of the data is closer to 0, so it makes learning for next layer easier. but for output layer we use sigmoid func as it is between 0 & 1 which is what we want.

for Relu (Rectified linear unit)



Leaky Relu:



why need a non-linear activation func.

if we use linear func as identity func, it will output a linear func rendering one hidden layer useless.

slopes:

sigmoid func: $g'(z) = g(z)[1 - g(z)]$

tanh(z) func: $g'(z) = 1 - (\tanh(z))^2$

$= 1 - [g(z)]^2$

Relu func:

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Gradient Descent on Neural networks

Parameters: $w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}$ $n_x = n^{[0]}, n^{[1]}, n^{[2]}$

$(n^{[1]}, n^{[0]})$ $(n^{[1]}, 1)$ $(n^{[2]}, n^{[1]})$ $(n^{[2]}, 1)$

cost function: $J(w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}, y)$

$\hat{y} = a^{[2]}$

Gradient descent:

Repeat {

compute predicts $(\hat{y}^{(i)}, i = 1, \ldots m)$

$dw^{[1]} = \frac{dJ}{dw^{[1]}}$, $db^{[1]} = \frac{\partial J}{\partial b^{[1]}}$, ...

$w^{[1]} := w^{[1]} - \alpha\, dw^{[1]}$

$b^{[1]} := b^{[1]} - \alpha\, db^{[1]}$

formula for computing derivatives,

forward propagation:

$$z^{[1]} = w^{[1]}x + b^{[1]}$$
$$A^{[1]} = g^{[1]}(z^{[1]})$$
$$z^{[2]} = w^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = g^{[2]}(z^{[2]}) = \sigma(z^{[2]})$$

Back propagation:

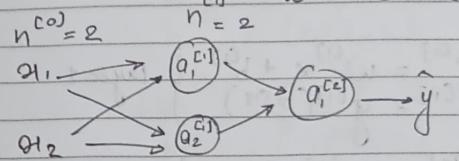
$$dz^{[2]} = A^{[2]} - Y \qquad Y = [y^{(1)}\ y^{(2)} \dots y^{(m)}]$$
$$dw^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$$
$$db^{[2]} = \frac{1}{m} np.sum(dz^{[2]}, axis=1, keepdims=true)$$
$$dz^{[1]} = \underbrace{w^{[2]T} dz^{[2]}}_{(n^{[1]}, m)} * \underbrace{g^{[1]'}(z^{[1]})}_{(n^{[1]}, m)}$$

element wise product

$$dw^{[1]} = \frac{1}{m} dz^{[1]} x^T$$
$$db^{[1]} = \frac{1}{m} np.sum(dz^{[1]}, axis=1, keepdims=true)$$

Random Initialization

$w^{[2]} = [0, 0]$



initialize to 0.

$$w^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \qquad b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$a_1^{[1]} = a_2^{[1]} \qquad dz_1^{[1]} = dz_2^{[1]}$$

∴ nodes in hidden layer will compute the same thing.

$$w^{[1]} = w^{[1]} - \alpha dw^{[1]}$$

after the iteration we get 1st & 2nd row as same.

Deep L-layer Neural Network.

neural network with multiple hidden layers.

Forward Propagation in a deep Network

x: $z^{[1]} = w^{[1]} x + b^{[1]}$
$a^{[1]} = g^{[1]}(z^{[1]})$ } layer 1

$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$
$a^{[2]} = g^{[2]}(z^{[2]})$ } layer 2

⋮

general:

$$z^{[l]} = w^{[l]} a^{[l-1]} + b^{[l]}$$
$$a^{[l]} = g^{[l]}(z^{[l]})$$

vectorized:

$$Z^{[1]} = W^{[1]} A^{[0]} + b^{[1]}$$
$$A^{[1]} = g^{[1]}(Z^{[1]})$$
$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[1]}$$
$$A^{[2]} = g^{[2]}(Z^{[2]})$$
$$\hat{y} = g(Z^{[4]}) = A^{[4]}$$

} for $l = 1 \ldots 4$

checking dimensions:

$w^{[l]} : (n^{[l]}, n^{[l-1]})$
$b^{[l]} = (n^{[l]}, 1)$

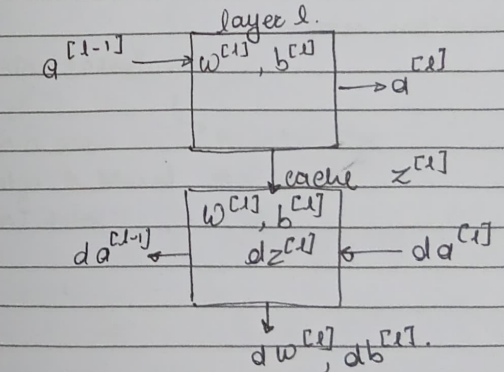
$dw^{[l]} : (n^{[l]}, n^{[l-1]})$
$db^{[l]} : (n^{[l]}, 1)$

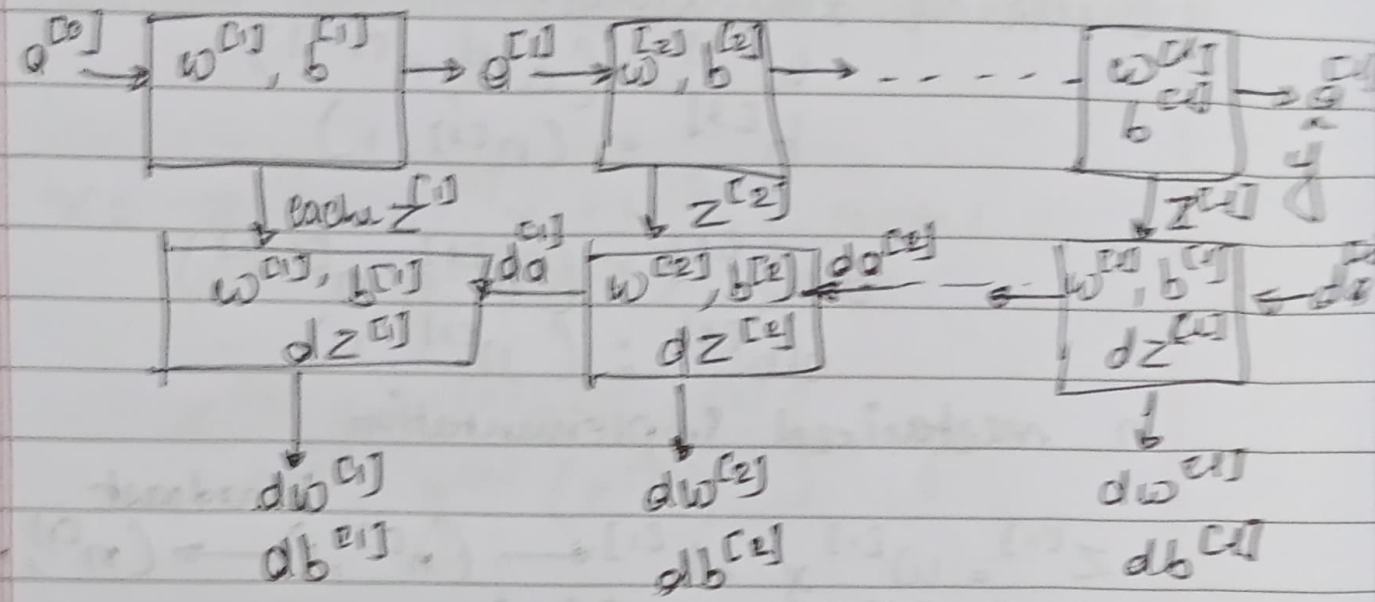
in vectorized implementation,

broadcast

$Z^{[l]} = W^{[l]} \times A^{[l-1]} + b^{[l]} \leftarrow (n^{[l]}, 1) \rightarrow (n^{[l]}, m)$
$(n^{[l]}, m) \quad (n^{[l]}, n^{[l-1]}) \quad (n^{[l-1]}, m)$

Building Blocks of Deep NN





$$w^{[1]} := w^{[1]} - \alpha\, dw^{[1]}$$
$$b^{[1]} := b^{[1]} - \alpha\, db^{[1]}$$